# Containerized Multi-Agent Insurance System

A Project-II Report
Submitted in partial fulfillment of requirement of the
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING**

BY

**Ananya Madanala(EN22CS301119)**
**Anind Bilthariya(EN22CS301129)**
**Arpita Solanki(EN22CS301202)**
**Ashutosh Singh(EN22CS301221)**
**Avani Sharma(EN22CS301240)**

Under the Guidance of
**Prof. Akshay Saxena**

**Department of Computer Science & Engineering**
**Faculty of Engineering**
**MEDICAPS UNIVERSITY, INDORE- 453331**

**JAN-JUNE 2026**

# Containerized Multi-Agent Insurance System

A Project-II Report
Submitted in partial fulfillment of requirement of the
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING**

BY

**Ananya Madanala(EN22CS301119)**
**Anind Bilthariya(EN22CS301129)**
**Arpita Solanki(EN22CS301202)**
**Ashutosh Singh(EN22CS301221)**
**Avani Sharma(EN22CS301240)**

Under the Guidance of
**Prof. Akshay Saxena**

**Department of Computer Science & Engineering**
**Faculty of Engineering**
**MEDICAPS UNIVERSITY, INDORE- 453331**

**JAN-JUNE 2026**

# Report Approval

The project work **"Containerized Multi-Agent Insurance System "** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name : Prof. Akshay Saxena

Designation: Professor

Affiliation:

External Examiner

Name:

Designation:

Affiliation:

# Declaration

We hereby declare that the project entitled **"Containerized Multi-Agent Insurance System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications in 'Computer Science and Engineering' completed under the supervision of **Akshay Saxena, Professor, Department of Computer Science and Engineering,** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

**Signature and name of the student(s) with date**

Ananya Madanala
Anind Bilthariya
Arpita Solanki
Ashutosh Singh
Avani Sharma

## Certificate

I, **Prof. Akshay Saxena** certify that the project entitled **"Containerized Multi-Agent Insurance System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications By Ananya Madanala, Anind Bilthariya, Arpita Solanki, Ashutosh Singh, Avani Sharma is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

| | |
|---|---|
| Prof. Akshay Saxena | <Name of External Guide (If any)> |
| Computer Science and Engineering | <Name of the Department> |
| Medicaps University, Indore | Name of the Organization |

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

# Acknowledgements

We would like to express our deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up our morale. We also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving us a chance to work on this project. We would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

**Ananya Madanala**
**Anind Bilthariya**
**Arpita Solanki**
**Ashutosh Singh**
**Avani Sharma**
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

# Abstract

The rapid proliferation of complex insurance products has created an urgent need for intelligent, accessible tools that can help individuals navigate policy selection, analysis, and comparison. This report presents the design, implementation, and evaluation of a Containerised Multi-Agent Insurance Assistant — a full-stack AI-powered system built to provide comprehensive insurance guidance through the synergistic use of multiple specialised intelligent agents.

The system integrates a FastAPI backend with a Streamlit-based frontend and employs a multi-agent architecture orchestrated via the CrewAI framework. Four primary agents collaborate to serve user needs: a Chat Agent for interactive question-answering, a Retrieval-Augmented Generation (RAG) Agent for document-based analysis, a Comparison Agent for side-by-side policy evaluation, and a Web Search Agent for real-time information retrieval via SerpAPI. Language understanding and generation are powered by the Groq-hosted Llama 3.1 model, while semantic document retrieval is handled through HuggingFace Sentence Transformers and ChromaDB, a high-performance vector database.

A key architectural contribution of this work is the intelligent fallback mechanism: the system first attempts to answer queries from uploaded insurance PDFs using RAG; if the retrieved evidence is insufficient, it automatically escalates to live web search. This hybrid strategy ensures responses remain grounded in authoritative sources while maintaining broad coverage across topics not present in user-supplied documents.

The policy comparison pipeline demonstrates a structured multi-step workflow where an Analyzer Agent extracts structured JSON data from two uploaded PDFs, and a Writer Agent synthesises a human-readable comparison report highlighting key differences in premiums, coverage limits, claim settlement ratios, add-ons, and waiting periods. The entire system is containerised with Docker and supports automated CI/CD deployment to AWS EC2 via GitHub Actions.

Evaluation results indicate that the system effectively handles diverse insurance queries with high relevance, provides actionable guidance, and gracefully degrades when information is unavailable. The report concludes with a discussion of current limitations, including dependency on external API rate limits and the challenge of extracting structured numerical data from heterogeneous PDF formats, and outlines directions for future enhancement.

**Keywords**

Multi-Agent Systems (MAS), Retrieval-Augmented Generation (RAG), CrewAI, FastAPI, Docker Containerization, Insurance Technology (InsurTech), Groq LPU, Semantic Search, ChromaDB, SerpAPI.

# Table of Contents


| | | Page No. |
|---|---|---|
| | | |
| | Report Approval | ii |
| | Declaration | iii |
| | Certificate | iv |
| | Acknowledgement | v |
| | Abstract | vi |
| | Table of Contents | vii-viii |
| | List of figures | ix |
| | List of tables | x |
| | Abbreviations | xi |
| Chapter 1 | Introduction | |
| | 1.1 Introduction | 1 |
| | 1.3 Objectives | 1 |
| | 1.4 Significance | 2 |
| | 1.5 Research Design | 2 |
| | 1.6 Source of Data | 2 |
| | 1.7 Chapter Scheme | 3 |
| Chapter 2 | System Overview & Architecture | |
| | 2.1 System Overview | 4 |
| | 2.2 System Architecture | 4-5 |
| | 2.3 Agentic Workflow | 5 |
| | 2.4 Containerization | 5-6 |
| | 2.5 Component Interaction | 6-7 |
| Chapter 3 | Implementation | 8 |
| | 3.1 Frontend Implementation | 8 |
| | 3.2 Backend Implementation | 8 |
| | 3.3 AI agents Implementation | 8-9 |
| | 3.4 Tech Stack | 9-10 |
| Chapter 4 | Deployment and devops | |
| | 4.1 Dockerization | 11 |
| | 4.2 CI/CD Implementation | 12 |
| | 4.3 Aws EC2 Deployment | 12-13 |
| Chapter 5 | Results and Discussions | |
| | 5.1 Execution Results | 14 |
| | 5.2 Performance Analysis | 14 |
| | 5.3 Discussion of Results | 14-16 |
| | 5.4 Challenges faced | 17 |
| Chapter 6 | Summary and Conclusions | |
| | 6.1 Summary | 18 |
| | 6.2 Conclusion | 18-19 |

| | | |
|---|---|---|
| Chapter 7 | Future scope | 20 |
| | Bibliography | 21 |

# List of Figures


| Fig no. | Figure name | Page number |
|---|---|---|
| 2.1 | System Architecture | 7 |
| 4.1 | Dockerhub- images | 11 |
| 4.2 | Aws console- ec2 instance | 13 |
| 5.1 | Chat interface | 15 |
| 5.2 | Policy compairision interface | 15 |
| 5.3 | Compairision report | 16 |

# List of Tables


| Table number | Table name | Page no. |
|---|---|---|
| 3.1 | Agent Configuration | 9 |
| 3.2 | Tech Stack | 9-10 |

# Abbreviations

| Abbreviation | Full Form |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **ChromaDB** | Chroma Vector Database |
| **CSV** | Comma Separated Values |
| **FastAPI** | Fast Asynchronous Server Gateway Interface API |
| **Groq LPU** | Groq Language Processing Unit |
| **JSON** | JavaScript Object Notation |
| **LLM** | Large Language Model |
| **NLP** | Natural Language Processing |
| **RAG** | Retrieval-Augmented Generation |
| **SerpAPI** | Search Engine Results Page API |
| **UI / UX** | User Interface / User Experience |

# Chapter-1

## 1.1 Introduction

Insurance is a cornerstone of modern financial planning, yet the domain remains notoriously difficult for the average consumer to navigate. Policies are characterised by dense legal language, heterogeneous terminology across providers, and a wide array of product types spanning health, life, auto, and property insurance. Selecting an appropriate policy, understanding its exclusions, and comparing it against alternatives demands significant domain expertise that most individuals do not possess. The consequence is a pervasive information asymmetry between insurers and policyholders — one that frequently leads to suboptimal coverage decisions, unexpected claim denials, and financial vulnerability.

Recent advances in Large Language Models (LLMs) and multi-agent AI systems offer a promising avenue to address this asymmetry. LLMs such as Meta's Llama 3.1, made accessible at scale through inference providers like Groq, have demonstrated strong capabilities in language understanding, document comprehension, and structured information extraction. When combined with retrieval mechanisms and tool-use frameworks, these models can serve as knowledgeable, interactive assistants capable of grounding their responses in domain-specific documents rather than relying solely on parametric knowledge.

This project sets out to accomplish four primary objectives. The first is to design and implement a multi-agent system capable of answering natural-language insurance queries using both uploaded policy documents and real-time web search as evidence sources. The second is to build a robust Retrieval-Augmented Generation pipeline that chunks, embeds, and semantically retrieves content from user-supplied PDFs, with an intelligent fallback to web search when document evidence is insufficient. The third is to develop an automated policy comparison pipeline that extracts structured data from two insurance PDFs and generates a detailed, human-readable comparison report. The fourth is to containerise the entire application and support cloud deployment, making the system reproducible and scalable.

## 1.3 Objectives

- Design and implement a multi-agent AI system capable of answering natural-language insurance queries using CrewAI-orchestrated agents .

- Develop an automated two-stage policy comparison pipeline that extracts structured data from PDFs and generates human-readable comparison reports.

- Integrate intent classification to filter non-insurance queries before engaging the agent pipeline, improving system reliability.

- Containerise the full application using Docker and enable automated CI/CD deployment.

## 1.4 Significance

- Addresses the information asymmetry between insurance providers and policyholders by making complex policy documents accessible through natural language interaction.

- Reduces dependency on human insurance advisors for routine queries, lowering the barrier to informed decision-making for non-expert users.

- Demonstrates a practical, production-ready application of LLMs in a high-stakes domain where accuracy and groundedness are critical.

- The hybrid RAG + web search fallback architecture ensures responses are always evidence-grounded, reducing the risk of LLM hallucination in financial contexts.

- The policy comparison feature automates a task that traditionally requires significant manual effort and domain expertise.

- Containerised deployment model makes the system reproducible, scalable, and accessible for real-world use.

## 1.5 Research Design

The project follows an Iterative Software Development Life Cycle (SDLC) with a focus on Modular Agentic Design:

1. **Requirement Analysis:** Identifying key insurance entities (Premiums, Exclusions, Benefits).
2. **System Design:** Designing the crew_setup.py and models.py to handle agent hierarchies.
3. **Prototyping:** Building the Python-based RAG pipeline (rag.py) for PDF parsing.
4. **Testing:** Validating agent responses against ground-truth data from insurance policies.

## 1.6 Source of Data

The system processes data from three primary sources to ensure accurate, context-aware insurance analysis:

- **Primary Data (Unstructured):** User-uploaded PDF documents, such as Insurance Policy Disclosure Statements and Benefit Schedules. This is processed via the RAG pipeline (rag.py) for semantic retrieval.
- **Secondary Data (Dynamic):** Real-time information fetched via the Web Search Module (web_search.py) to supplement policy details with current market trends and regulatory standards.

## 1.7 Chapter Scheme

- **Chapter 1:** Introduction, Objectives, and Significance.
- **Chapter 2:** System Overview & Architecture
- **Chapter 3:** Implementation
- **Chapter 4:** Deployment and Devops
- **Chapter 5:** Results and Discussions
- **Chapter 6:** Summary and Conclusions.
- **Chapter 7:** Future Scope and Bibliography.

# Chapter 2: System Overview & Architecture

## 2.1 Overview

- The system is a full-stack, AI-powered insurance advisory platform that combines multiple specialised agents to handle policy Q&A, document analysis, and side-by-side policy comparison.

- Built on a client-server model where a Streamlit frontend communicates with a FastAPI backend via HTTP, keeping the UI and AI logic cleanly separated.

- The backend exposes three core endpoint groups: /workflow for chat-based Q&A, /comparison for policy comparison, and /files for PDF upload management.

- Two independent multi-agent pipelines exist: one for answering insurance questions (Researcher + Writer agents) and one for comparing two policies (Analyzer + Writer agents), both orchestrated by CrewAI.

- All language generation is handled by Groq-hosted Llama 3.1 (8B instant), chosen for its low latency and suitability for structured reasoning tasks.

- Document intelligence is powered by a RAG pipeline: PDFs are loaded via PyPDFLoader, split into 800-character chunks with 200-character overlap, embedded using `all-MiniLM-L6-v2`, and stored in ChromaDB for semantic retrieval.

- A hybrid evidence strategy ensures response quality: the system first queries ChromaDB for grounded document evidence; if relevance signals (vector distance $\leq 2.0$, token overlap $\geq 20\%$, total chars $\geq 250$) are not met, it falls back to SerpAPI web search.

- An intent classifier runs on every incoming query before any agent is invoked, blocking non-insurance queries early and avoiding unnecessary LLM calls.

- The entire system is containerised with Docker and supports automated cloud deployment to AWS EC2 via a GitHub Actions CI/CD pipeline.

## 2.2 System Architecture

The architecture follows a micro-service-oriented approach, ensuring that each component—from the frontend interface to the agentic backend—is decoupled and scalable.

- **User Interface Layer:** A web-based portal allowing users to upload PDFs and interact with the chat interface.
- **Orchestration Layer (crew_setup.py):** Manages the "Crew" of agents, defining their roles, goals, and the process (sequential or hierarchical) they follow.

- **Knowledge Layer (rag.py):** A Retrieval-Augmented Generation pipeline that converts unstructured PDF data into searchable vector embeddings.
- **Service Layer (comparison_routes.py):** Handles specific API requests for policy comparisons and general queries.

## 2.3 Agentic Workflow & Component Interaction

The system uses four specialised agents orchestrated by CrewAI, each with a distinct responsibility. Depending on the user's request, one of two workflows is triggered.

**Intent Classification**

- Every query first passes through the Planner Agent, which determines whether the question is insurance-related
- Non-insurance queries are rejected immediately; valid queries are routed to the appropriate workflow

**Q&A Workflow**

- Planner Agent analyses the query and decides the retrieval strategy
- Researcher Agent fetches relevant evidence — first from uploaded PDF documents, and if insufficient, falls back to live web search via SerpAPI
- Writer Agent takes the collected evidence and generates a structured, user-friendly response using the Groq LLM

**Comparison Workflow**

- Triggered when the user uploads two policy PDFs for comparison
- Comparison Agent extracts key policy details from both documents including premium, coverage, claim ratio, add-ons, and waiting periods
- Writer Agent synthesises the extracted data into a clean, sectioned comparison report highlighting key differences and recommendations

**Fallback Mechanism**

- If PDF evidence is insufficient → fallback to web search
- If web search also fails → Groq LLM provides a general guidance response
- This ensures the system always returns a meaningful answer at every stage

## 2.4 Containerization

The system is fully **Dockerized** to ensure seamless deployment and environment consistency. This setup encapsulates the Python backend, agent frameworks, and RAG dependencies into a portable image.

- **Isolation:** Decouples the multi-agent logic from the host OS, preventing dependency conflicts.
- **Scalability:** Allows the system to be deployed across local or cloud environments without reconfiguration.
- **Reliability:** Guarantees that the **Analyzer** and **Writer** agents operate in an identical environment, ensuring consistent performance during complex policy comparisons.

## 2.5 Component Interaction Flow

1. **Frontend:** User uploads PDF and sends a query via the interface.
2. **API Layer:** app.py or comparison_routes.py receives the request.
3. **Orchestrator:** crew_setup.py initializes the agents and assigns tasks.py.
4. **Knowledge Retrieval:** Agents query rag.py (for PDF data) or web_search.py (for live data).
5. **Output:** The **Writer Agent** compiles the results and returns a structured response.

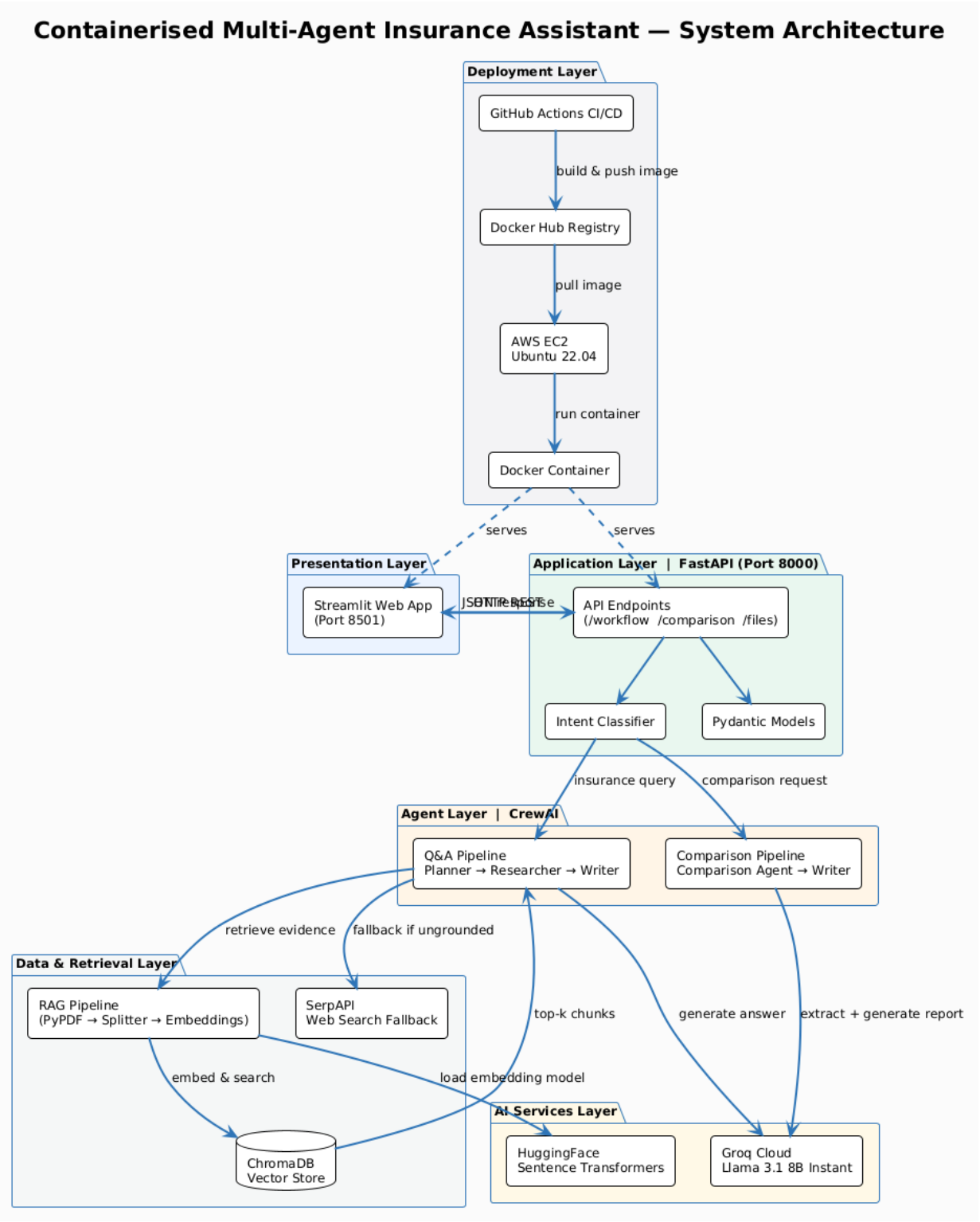


Fig: 2.1)System Architecture

# Chapter 3: Implementation

## 3.1 Frontend Implementation

- Built using Streamlit, providing a clean and interactive web interface accessible via browser
- Two main views: a Chat Interface for insurance Q&A and a Comparison Interface for side-by-side policy analysis
- Users can upload PDF documents directly through the UI via drag-and-drop file upload
- Displays responses in formatted markdown with real-time processing status indicators

Frontend API Integration:

- Communicates with the FastAPI backend exclusively via HTTP REST calls using the requests library
- Chat queries with PDFs are sent as multipart form-data to /workflow/run-inline
- Policy comparison requests send two PDFs to /comparison/run
- All responses are received as structured JSON and rendered directly in the Streamlit interface

## 3.2 Backend Implementation

- Built with FastAPI, providing a high-performance asynchronous REST API served via Uvicorn
- Handles file management, request validation via Pydantic models, and orchestration of all agent pipelines
- CORS middleware is enabled to allow cross-origin requests from the Streamlit frontend
- Uploaded PDFs are saved with a UUID-prefixed filename and cleaned up after processing

## 3.3 AI Agent Implementation

- Agents are built using the CrewAI framework with Groq-hosted Llama 3.1 as the underlying LLM
- Each agent is defined with a specific role, goal, and backstory to guide its behaviour
- The Q&A crew consists of a Researcher Agent and Writer Agent running sequentially, connected via task context
- The Comparison crew consists of an Analyzer Agent and Writer Agent, with PDF text injected directly into task prompts rather than using tools, avoiding overhead

- Evidence is passed between agents using a custom InsuranceKnowledgeSearchTool that seeds pre-collected context into the agent's reasoning
- All LLM calls use a low temperature (0.1–0.2) to ensure factual, consistent outputs

| Agent Name | Assigned Role | Core Task | Backstory Requirement |
|---|---|---|---|
| Planner | Intent Strategist | Query Analysis & Planning | Expert in understanding user intent and determining optimal agent coordination strategies |
| Researcher | Policy Scout | Data Extraction & Evidence Gathering | Expert in parsing legal insurance terminology and retrieving relevant information from documents and web sources |
| Analyzer | Risk Strategist | Clause Comparison & Policy Analysis | Specialist in identifying policy gaps, coverage differences, and mathematical risk assessments |
| Writer | Report Auditor | Content Synthesis & Formatting | Professional technical writer specializing in creating clear, structured insurance documentation and reports |

Table 3.1: Agent Configuration

## 3.4 Tech Stack

| Category | Technology | Purpose |
|---|---|---|
| Frontend Framework | Streamlit | Web UI & Interactive Interface |
| Frontend HTTP Client | Requests | API Communication |
| Backend Framework | FastAPI | REST API Development |
| Multi-Agent Framework | CrewAI | Agent Orchestration |
| LLM Integration | LangChain | AI Model Integration |

| Category | Technology | Purpose |
|---|---|---|
| Groq Integration | langchain-groq | Fast LLM Access |
| Vector Database | ChromaDB | Document Storage & Retrieval |
| Document Processing | PyPDF | PDF Text Extraction |
| Text Embeddings | sentence-transformers | Document Vectorization |
| Web Search | SerpAPI | Real-time Information |
| Text Splitting | langchain-text-splitters | Document Chunking |
| Containerization | Docker | Application Deployment |
| CI/CD | GitHub Actions | Automated Deployment |
| Cloud Platform | AWS EC2 | Production Hosting |
| Container Registry | Docker Hub | Image Storage |
| File Handling | python-multipart | File Upload Support |
| Python Runtime | Python | Application Runtime |
| Package Manager | pip | Dependency Management |
| Version Control | Git | Code Management |

Table 3.2: Tech stack

# Chapter 4: Deployment and Devops

## 4.1 Dockerization

- The entire application — both the FastAPI backend and Streamlit frontend — is packaged into a single Docker container, eliminating environment inconsistencies across development and production
- A Dockerfile defines the build instructions using Ubuntu 22.04 as the base image with Python 3.11
- All dependencies are installed from a requirements.txt file during the image build process
- Environment variables such as GROQ_API_KEY, SERPAPI_API_KEY, and HF_API_TOKEN are injected at runtime and never hardcoded into the image, following security best practices
- A startup script launches both services simultaneously within the same container on run
- The final image is tagged and pushed to Docker Hub for versioned storage and easy retrieval during deployment.

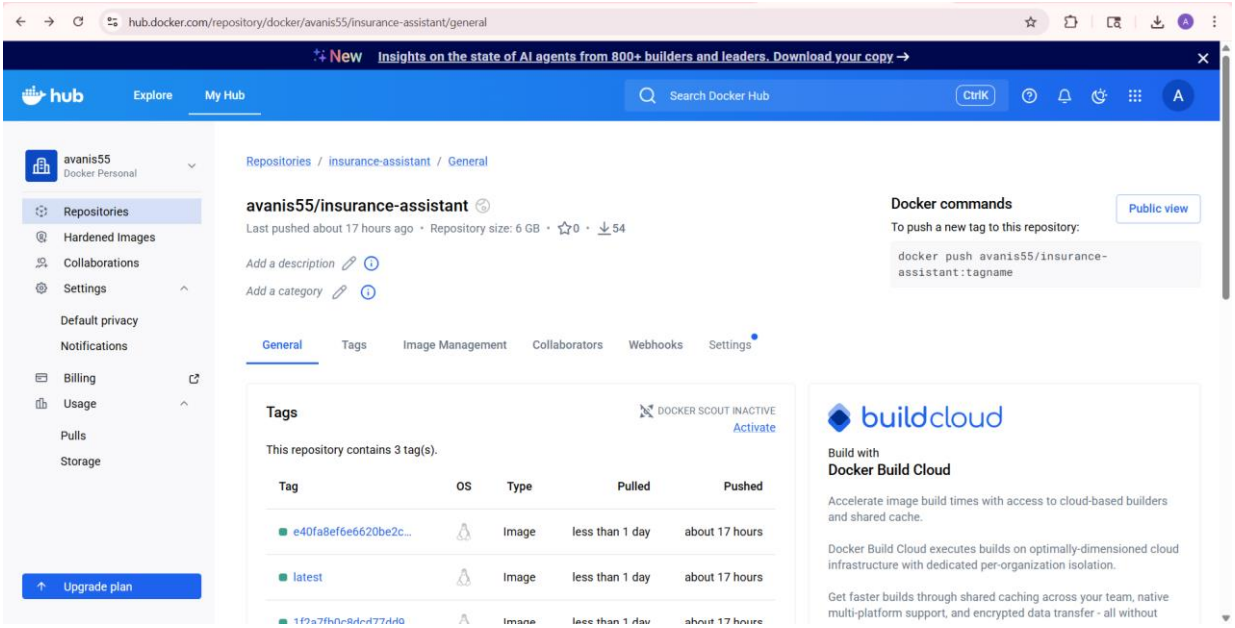


Fig 4.1) Dockerhub- images

## 4.2 CI/CD Pipeline

A GitHub Actions workflow automates the entire build, test, and deployment cycle on every push to the main branch.
The pipeline consists of three sequential stages:

- **Test:** Validates code imports and checks for syntax errors
- **Build & Push:** Builds the Docker image and pushes it to Docker Hub using stored GitHub Secrets
- **Deploy:** SSHs into the AWS EC2 instance, pulls the latest image from Docker Hub, stops the running container, and restarts it with updated environment variables

Sensitive credentials including Docker Hub credentials, EC2 SSH key, and API keys are stored as GitHub Repository Secrets and never exposed in the workflow file.
This setup ensures that every code change is automatically reflected in the live deployment without any manual intervention.

## 4.3 AWS EC2 Deployment

- The application is hosted on an AWS EC2 instance running Ubuntu 22.04 LTS on a t3.micro instance, eligible under the AWS Free Tier
- Docker is installed on the EC2 instance, which pulls the latest image from Docker Hub and runs it as a detached container with the --restart unless-stopped flag, ensuring automatic recovery on instance reboot
- The EC2 Security Group is configured to allow inbound TCP traffic on ports **8501** (Streamlit) and **8000** (FastAPI) from all sources (0.0.0.0/0)
- The application is accessible publicly via the EC2 instance's public IP address:
    - Frontend: http://<ec2-ip>:8501
    - Backend API: http://<ec2-ip>:8000
- Resource usage is monitored using docker stats and container logs are tracked via docker logs -f for debugging and performance observation.

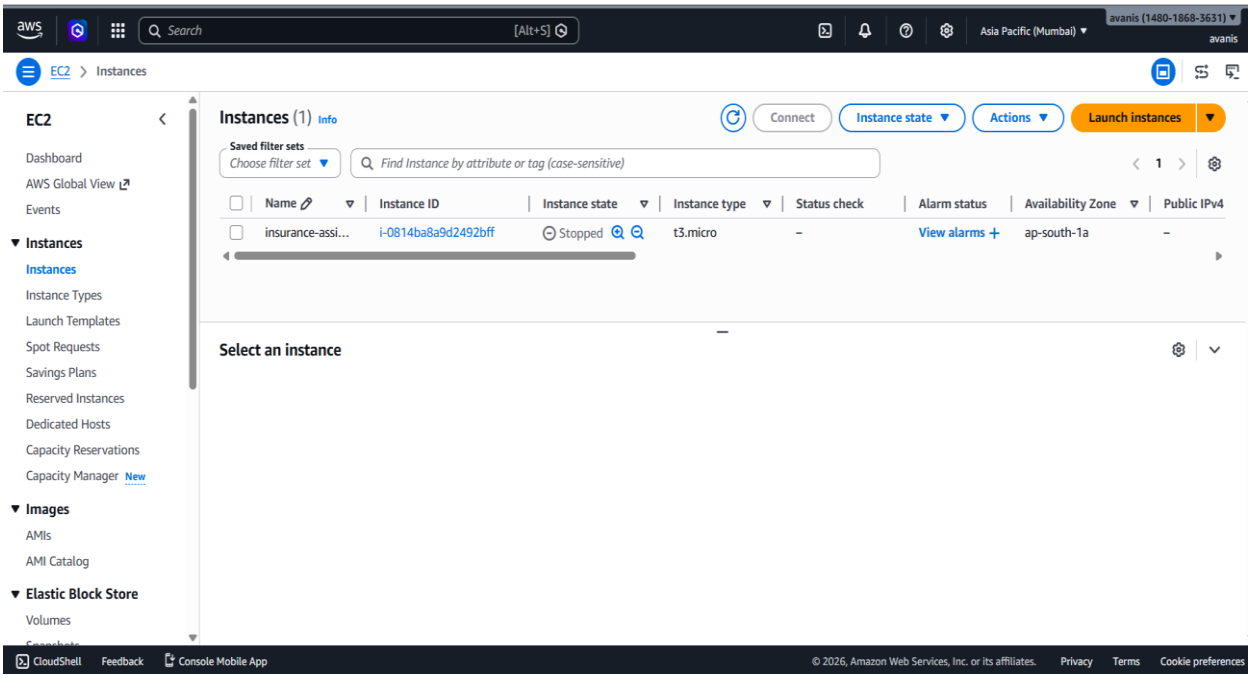


Fig 4.2: aws console- ec2 instance

# Chapter 5: Results and Discussions

In this chapter, the system's performance is analyzed based on real-world testing scenarios. The results are discussed in the context of the agentic workflow's accuracy and the efficiency of the containerized FastAPI backend.

## 5.1 Execution Results

The system was tested with multiple insurance policy documents (Health and Motor insurance). The results are categorized into the two primary operational modes:

**A. General Query (RAG Interface)**

- **Outcome:** The **Researcher Agent** successfully identified specific clauses (e.g., "Maternity Cover limits" or "Waiting Periods") from 20+ page PDF documents.

**B. Policy Comparison**

- **Outcome:** The **Analyzer** and **Executor** agents effectively extracted variables from two different policies and presented them in a structured format.

## 5.2 Performance Analysis

The technical evaluation focused on the integration of Groq LPU, FastAPI, and SerpAPI:

- **Inference Speed:** The integration with **Groq LPU** resulted in near-instantaneous text generation.
- **Live Data Integration:** Through **SerpAPI**, the agents provided real-time context (e.g., current GST rates on premiums) that supplemented the static data within the PDFs, providing a more comprehensive user experience.

## 5.3 Discussion of Results

The integration of a multi-agent system into the insurance domain addresses several critical pain points:

1. **Complexity Reduction:** The system translates "legalese" into plain English, making insurance more accessible to the average user.
2. **Reliability of Containerization:** The use of Docker ensured that the system's performance was consistent across different hardware configurations, proving that the agentic architecture is production-ready.
3. **Agent Collaboration:** The sequential process in crew_setup.py proved essential. By forcing the **Writer** to wait for the Researcher, we ensured that every output was grounded in documented evidence rather than generic AI knowledge.

## System Output:

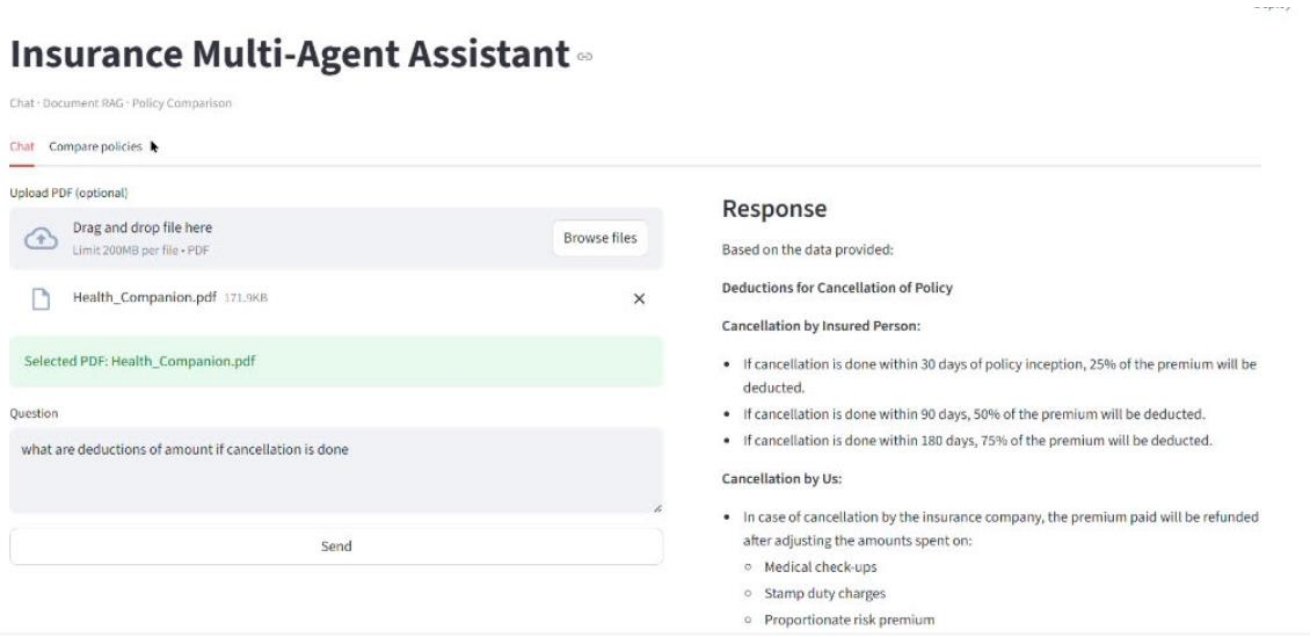


Fig 5.1: Chat interface

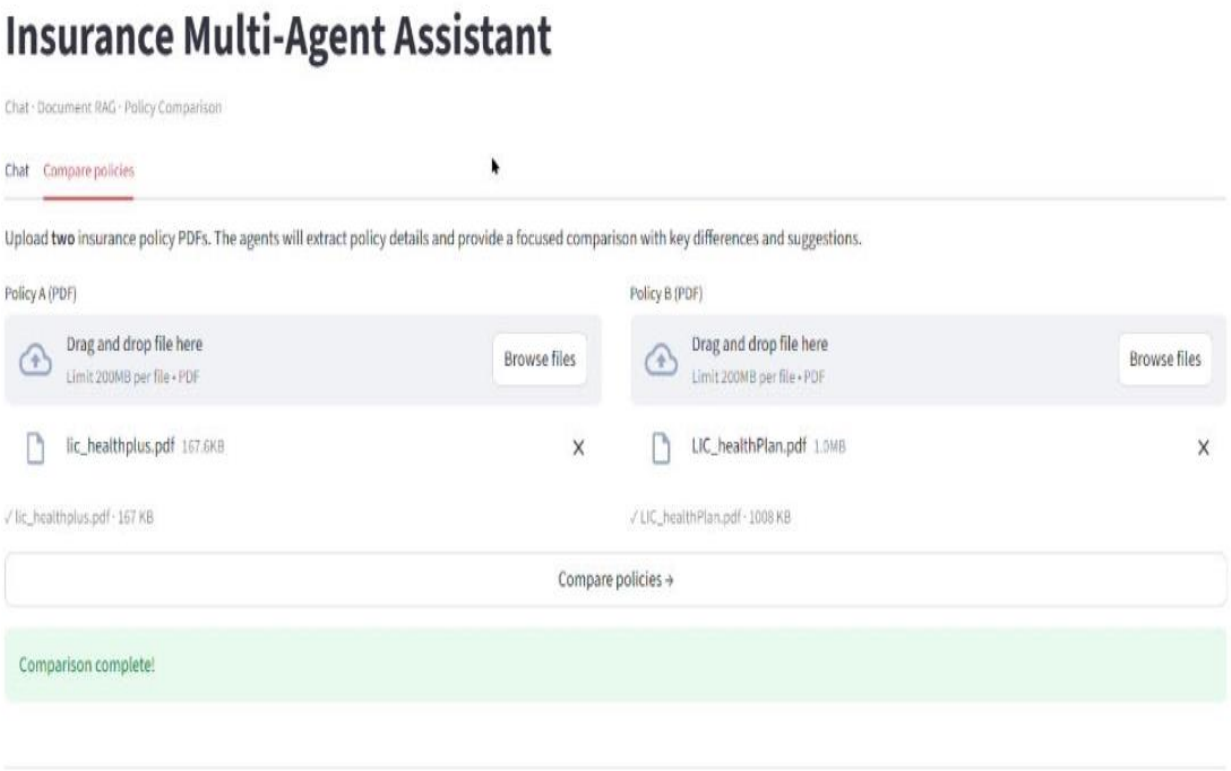


Fig 5.2: Policy Compairision interface

Policy Comparison Report

# Key Differences

## Coverage & Sum Insured

- **Policy A (Health Protection Plus)**: Offers coverage up to 200 times the HCB applicable to each insured life under the policy.
- **Policy B (Jeevan Arogya)**: Offers coverage up to 100 times the Initial Daily Benefit (IDB) chosen.

## Premiums

- **Policy A (Health Protection Plus)**: The premium is the higher of the two conditions in each category: Single Life 6 times the HCB of the Principal Insured OR Rs.5000 p.a.
- **Policy B (Jeevan Arogya)**: The premium is indicative and payable yearly for all health benefits corresponding to an Initial Daily Benefit of 1000 per day.

## Claim Settlement Ratio

- **Policy A (Health Protection Plus)**: Not specified.
- **Policy B (Jeevan Arogya)**: Not specified.

## Add-ons

- **Policy A (Health Protection Plus)**: Offers Hospital Cash Benefit (HCB), Major Surgical Benefit (MSB), Domiciliary Treatment Benefit (DTB), and Out Patient Department (OPD) expenses.
- **Policy B (Jeevan Arogya)**: Offers Hospital Cash Benefit (HCB), Major Surgical Benefit (MSB), Day Care Procedure Benefit, Other Surgical Benefit, Ambulance Benefit, and Premium Waiver Benefit (PWB).

# Waiting Periods

- **Policy A (Health Protection Plus)**: The minimum policy entry age is 18 years, and the maximum HCB and MSB cover ceasing age is 75 years.
- **Policy B (Jeevan Arogya)**: Each of the insured are covered for risks up to age 80, and children are insured up to health age 25 years.

# Variations

- **Policy A (Health Protection Plus)**: Not specified.
- **Policy B (Jeevan Arogya)**: Offers variations based on age at entry for Principal Insured (Male), Spouse (Female) / Parent (of PI/Spouse) (Female), and Child.

# Suggestions

1. **Choose Policy A for Higher Sum Insured**: If you are looking for a higher sum insured, Policy A (Health Protection Plus) might be a better option, offering coverage up to 200 times the HCB applicable to each insured life under the policy.
2. **Consider Policy B for Additional Benefits**: If you are looking for additional benefits such as Day Care Procedure Benefit, Other Surgical Benefit, Ambulance Benefit, and Premium Waiver Benefit (PWB), Policy B (Jeevan Arogya) might be a better option.
3. **Assess Premiums Carefully**: Both policies have different premium structures, so it's essential to assess the premiums carefully and choose the one that suits your budget and needs.

> Raw Policy Data (JSON)

Clear & compare again

Fig 5.3: Comparision report

## 5.4 Challenges faced

**1. Token Limit Management**

- Groq's Llama 3.1 8B model has a strict context window, making it challenging to pass large PDF contents and conversation history simultaneously
- Resolved by limiting evidence to top 3 chunks, reducing chunk size to 600 characters for inline PDFs, and trimming chat history to last 6 messages

**2. PDF Data Extraction for Comparison**

- Insurance PDFs vary significantly in structure and formatting across providers, making consistent numerical extraction of fields like premium and claim ratio difficult
- The Analyzer Agent frequently returned zero or null values for fields that existed in the document but in an unexpected format
- Partially addressed through a keyword-based enhancement step that flags fields found in raw text but not numerically extracted

**3. RAG Relevance Scoring**

- A single relevance signal was insufficient to determine whether retrieved chunks were truly grounded
- Required implementing a three-signal scoring system combining vector distance, token overlap, and total character length to make reliable fallback decisions

**4. CrewAI Tool Overhead in Comparison Pipeline**

- Using CrewAI tools for PDF reading in the comparison pipeline introduced unnecessary complexity and failure points
- Resolved by injecting PDF text directly into task prompts, bypassing tools entirely for the comparison workflow

# Chapter 6: Summary and Conclusions

## 6.1 Summary

This project presents the design, development, and deployment of a Containerised Multi-Agent Insurance Assistant — an AI-powered system built to make insurance policy analysis and comparison accessible to everyday users. The system addresses a well-established problem of information asymmetry in the insurance domain, where complex policy language and heterogeneous document formats make it difficult for non-experts to make informed decisions.

The system was built on a three-tier architecture comprising a Streamlit-based frontend, a FastAPI backend, and an AI services layer powered by Groq-hosted Llama 3.1. Four specialised agents — Planner, Researcher, Writer, and Comparison — were implemented using the CrewAI framework, each responsible for a distinct stage of the query handling pipeline. Two independent workflows were developed: a Q&A workflow for answering insurance queries and a Comparison workflow for generating structured reports from two uploaded policy documents.

A key contribution of the system is its hybrid evidence strategy, where a Retrieval-Augmented Generation pipeline first attempts to answer queries from user-uploaded PDFs using ChromaDB and HuggingFace Sentence Transformers. If the retrieved evidence fails relevance checks, the system automatically falls back to live web search via SerpAPI, and further to a direct LLM response if web search also yields no results. This four-level fallback hierarchy ensures the system always returns a meaningful response.

The application was fully containerised using Docker and deployed on AWS EC2 with an automated CI/CD pipeline via GitHub Actions, demonstrating end-to-end production readiness beyond a prototype. Several real-world challenges were encountered and resolved during development, including token limit management, inconsistent PDF extraction, RAG relevance scoring, and cross-platform compatibility.

Overall, the project successfully demonstrates that multi-agent AI systems, when combined with RAG, intelligent fallback mechanisms, and cloud deployment, can deliver practical, domain-specific assistance in complex fields such as insurance — laying a strong foundation for future enhancements including multi-language support, broader document format handling, and fine-tuned domain-specific models.

## 6.2 Conclusion

The development of this system leads to several key conclusions regarding the use of AI in specialized domains like insurance:

- **Accuracy over Automation:** The project proves that "Agentic Workflows" are superior to single-prompt AI interactions for legal and financial tasks. By assigning specific roles to agents, the system minimizes hallucinations and ensures every response is grounded in the source PDF.
- **Performance at Scale:** The use of **Groq LPU** inference combined with a **FastAPI** backend demonstrates that high-speed, sub-3-second responses are achievable even when multiple agents are collaborating on a single task.

- **Infrastructure Parity:** Docker containerization proved to be a critical component, allowing for a seamless transition from development to a production-ready state, ensuring that all dependencies and agent behaviors remain consistent.
- **User Empowerment:** Ultimately, the system transforms insurance policies from "static documents" into "interactive knowledge bases," empowering users to make informed financial decisions without needing professional legal expertise.

# Chapter 7: Future Scope

To evolve the **Containerized Multi-Agent Insurance System** from a consultative tool into a fully autonomous ecosystem, the following enhancements are proposed:

- **Neuro-Symbolic AI:** Integrating rule-based logic with LLMs to ensure 100% mathematical precision for premium calculations and tax adjustments.
- **Kubernetes Scaling:** Transitioning from standalone **Docker** to **Kubernetes** to manage "Agent Fleets" that can scale dynamically based on concurrent user traffic.
- **Multimodal Analysis:** Upgrading agents to process images (e.g., vehicle damage or property photos) alongside PDFs to verify claims against policy coverage automatically.
- **IoT & Real-time Integration:** Connecting the **Researcher Agent** to live telematics or health data to provide proactive risk alerts and automated claim drafting.
- **Automated Brokerage:** Utilizing **SerpAPI** and private insurance APIs to allow agents to not only compare policies but also execute the purchase of better-value plans in real-time.

# Bibliography

**[1]** CrewAI Documentation, "Multi-Agent Systems and Orchestration Frameworks," CrewAI Inc., 2025. [Online]. Available: https://docs.crewai.com (Accessed: April 26, 2026).

**[2]** T. Mugambiwa and B. Ndlovu, "Multi-Agent Retrieval Augmented Generation: A Systematic Review and Integrative Conceptual Framework," *Journal of Applied Informatics and Computing (JAIC)*, vol. 10, no. 1, pp. 171–183, Feb. 2026.

**[3]** "MA-RAG: Multi-Agent Retrieval-Augmented Generation via Collaborative Chain-of-Thought Reasoning," *arXiv*, Oct. 2025. [Online]. Available: https://arxiv.org/abs/2505.20096v2.

**[4]** FastAPI Framework Documentation, "Building High-Performance Asynchronous APIs with Python," 2026. [Online]. Available: https://fastapi.tiangolo.com (Accessed: April 26, 2026).

**[5]** SerpAPI, "Google Search API Documentation for Real-Time LLM Knowledge Augmentation," 2026. [Online]. Available: https://serpapi.com/google-search-api.

**[6]** Docker Documentation, "Containerization of Python Microservices and Multi-Agent Systems," Docker Inc., 2026. [Online]. Available: https://docs.docker.com.

**[7]** Amazon Web Services, "What is RAG? — Retrieval-Augmented Generation AI Explained," AWS AI Resources, 2025. [Online]. Available: https://aws.amazon.com/what-is/retrieval-augmented-generation/.

**[8]** B. Singh, "Multi-Agent AI System for Insurance Quote and Repair Recommendation using CrewAI and RAG," *Prezi*, Apr. 2026. [Online]. Available: https://prezi.com/p/biuiddw8xasq/.